Dispensa unica 3INF — lezioni ed esercizi fino al 30/10/2026

Versione 1.0 — 30/10/2026

Classe 3INF · Laboratorio di informatica · a.s. 2026/2027. Una sola dispensa, uguale per tutti: le lezioni dal 23/09 al 29/10/2026 in ordine di data (l'ultima lezione della 3INF prima del 30/10 è giovedì 29/10), con la teoria in breve, le foto della lavagna, le righe da ricopiare sul quaderno e il testo completo dei compiti dati su Classroom. Il PDF serve per leggere e stampare; questo file MD si dà alla propria AI (Gemini, NotebookLM), che lo spiega con parole semplici e lo traduce nella propria lingua.

---

00 Come si usa questa dispensa

1. Ogni capitolo è **un giorno di lezione**, in ordine di data. Dentro trovi sempre le stesse parti:
   1.1 **Quando**: data, ore e materia;
   1.2 **Che cosa abbiamo fatto**;
   1.3 **La teoria**, in breve ma completa, con le foto della lavagna quando ci sono;
   1.4 **Da ricopiare sul quaderno**, quando c'era qualcosa da ricopiare;
   1.5 **Esercizi e compiti**: il titolo come su Classroom, il testo del compito, il link e che cosa si consegna.
2. In fondo ci sono la **tabella di tutti i compiti** (capitolo 18) e il **glossario** (capitolo 19).
3. Hai perso una lezione? Leggi il suo capitolo, poi apri il compito su Classroom e fallo: i compiti restano aperti come recupero, salvo diversa indicazione del prof.
4. Non capisci una parola? Dai questo file alla tua AI e chiedile: «Spiegami il capitolo 11 con un esempio, nella mia lingua». L'AI aiuta a **capire**, non a fare il lavoro al posto tuo: la prova che hai capito è saperlo spiegare a voce, con parole tue.
5. I link a Classroom si aprono solo con l'account della scuola. I link che cominciano con `nicolaregge-pulse.github.io` sono le pagine delle lezioni e si aprono da qualsiasi browser; molte hanno anche le righe in bangla.
6. Nei testi dei compiti gli orari di consegna («entro le 13:00») valevano per quel giorno: oggi servono solo a capire come era organizzata la lezione.
7. **Privacy:** il tuo sito è pubblico. Sul sito solo nome o soprannome: niente cognome, telefono, email, indirizzo, foto del viso.

> Il metodo del corso in tre parole: **vinci subito** (un primo risultato in pochi minuti), **fallo tuo** (scegli qualcosa di tuo: nome, colore, passione), **mostralo** (fallo vedere al compagno, anche sul telefono). Sbagliare è normale: succede a tutti i programmatori, anche ai professionisti.

01 Mercoledì 23/09/2026 — Presentazione dell'anno

01.1 Quando

1. Mercoledì 23/09/2026, 2 ore, Laboratorio.

01.2 Che cosa abbiamo fatto

1. Presentazione dell'anno scolastico 2026/27: argomenti, regole, modo di lavorare.

01.3 La teoria

1. **Che cosa facciamo quest'anno:** Git e GitHub, siti web con HTML e CSS pubblicati su Internet, il lavoro in squadra come in un'azienda di software (versioni, Pull Request, Agile), la programmazione con Lazarus e il confronto con Java.
2. **Come lavoriamo:** un passo alla volta, con un risultato concreto da vedere subito; consegne e materiali su **Google Classroom**; i progetti su **GitHub**.
3. **Regola d'oro a scuola:** non si installa niente. Si usano il browser, il Blocco note e i programmi portabili (che partono senza installazione).
4. **La regola dell'AI:** l'intelligenza artificiale aiuta a capire, non a saltare il pensiero.

01.4 Esercizi e compiti

1. Nessun compito su Classroom.

02 Venerdì 25/09/2026 — Le utenze Git e che cos'è Git

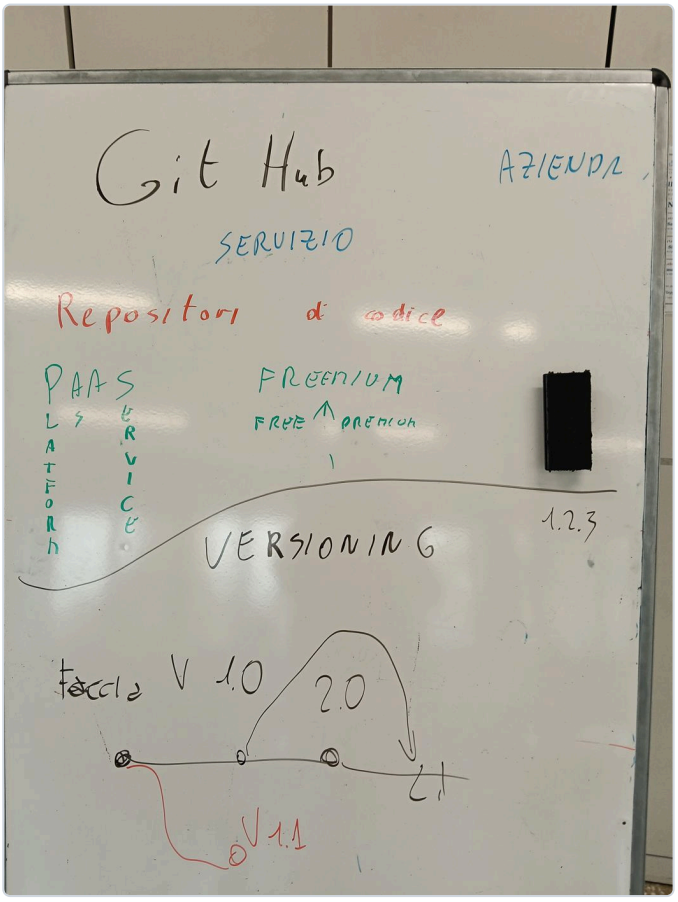
02.1 Quando

1. Venerdì 25/09/2026, 2 ore, Laboratorio.

02.2 Che cosa abbiamo fatto

1. Creazione dell'account GitHub con l'email della scuola; che cos'è Git; alla lavagna lo schema «GitHub: azienda, servizio, PaaS, Freemium, versioning».

02.3 La teoria



La lavagna del 25/09: GitHub è un'azienda e un servizio, tiene repository di codice; PaaS e Freemium; il versioning 1.2.3 e il grafo con V 1.0, V 1.1 e 2.0

1. **Git** è un **sistema di controllo di versione**: tiene traccia di tutte le modifiche ai file di un progetto, nel tempo.
2. **Commit:** una fotografia salvata delle modifiche, con un **messaggio** che dice che cosa hai cambiato.
3. **Repository** (repo): la cartella del progetto con tutti i file e la loro **storia completa**.
4. **A che cosa serve:** tornare a una versione precedente, sapere chi ha cambiato che cosa e quando, lavorare in gruppo senza sovrascriversi.
5. **Git e GitHub:** Git è lo strumento che tiene la storia (sul tuo computer); GitHub è il **sito** che ospita i repository online e permette di condividere e collaborare.
6. **Branch** (ramo): una linea di lavoro parallela, per provare cose nuove senza rovinare il progetto principale; poi si uniscono con un **merge**.

7. **GitHub sulla lavagna:** è un'**azienda** (GitHub, di Microsoft) e un **servizio**; tiene **repository di codice**; è una piattaforma **PaaS** (Platform as a Service) e si paga in **Freemium** (base gratis, extra a pagamento). Sotto, il **versioning**: i numeri 1.2.3 e un grafo con un ramo che si stacca (V 1.1) e la versione 2.0.

02.4 Esercizi e compiti

1. **GitHub**
   1.1 Testo del compito (il foglio «Compito: Git e Versioning»): obiettivo, dimostrare che hai capito che cos'è Git e come funzionano le versioni, con parole tue. Conta la comprensione, non le frasi perfette.
   1.2 Ripasso lampo: Git è la macchina del tempo del progetto (salva le modifiche, i commit, e ti fa tornare indietro); Git è lo strumento, GitHub il sito-servizio (PaaS, Freemium); repository, branch, merge; le versioni si scrivono v1.2.3 e una versione pubblicata non si riusa più.
   1.3 Apri un documento nuovo: nel browser vai su `docs.new` e dai un titolo, per esempio «Git e Versioning» con il tuo nome. Rispondi alle tre parti.
   1.4 Parte 1, Git con parole tue: che cos'è Git e a che problema serve (2-3 frasi); spiega con una frase ciascuno repository, commit, branch, merge; qual è la differenza tra Git e GitHub.
   1.5 Parte 2, versioning: un gioco è alla versione v1.2.0. Scrivi quale sarà la nuova versione e perché se correggi un piccolo bug; se aggiungi un nuovo livello e le cose vecchie funzionano ancora; se riscrivi tutto da capo con un motore nuovo, incompatibile con il vecchio. Poi guarda il grafo del foglio: quanti branch vedi oltre al main, quanti merge, a che cosa serviva il ramo rosso.
   1.6 Parte 3, la tua esperienza: che cosa hai fatto oggi con GitHub, che cosa è stato facile e che cosa difficile; un esempio tuo (una tesina, delle foto, un progetto) in cui sarebbe utile avere la storia delle versioni.
   1.7 Quando hai finito, consegna il documento su Classroom. Conta soprattutto saperlo spiegare a voce.
   1.8 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/MTYyNzIyOTgzNjk5/details
   1.9 Che cosa si consegna: il Documento con le tre parti.

03 Lunedì 28/09/2026 — Git e GitHub; i tipi di file

03.1 Quando

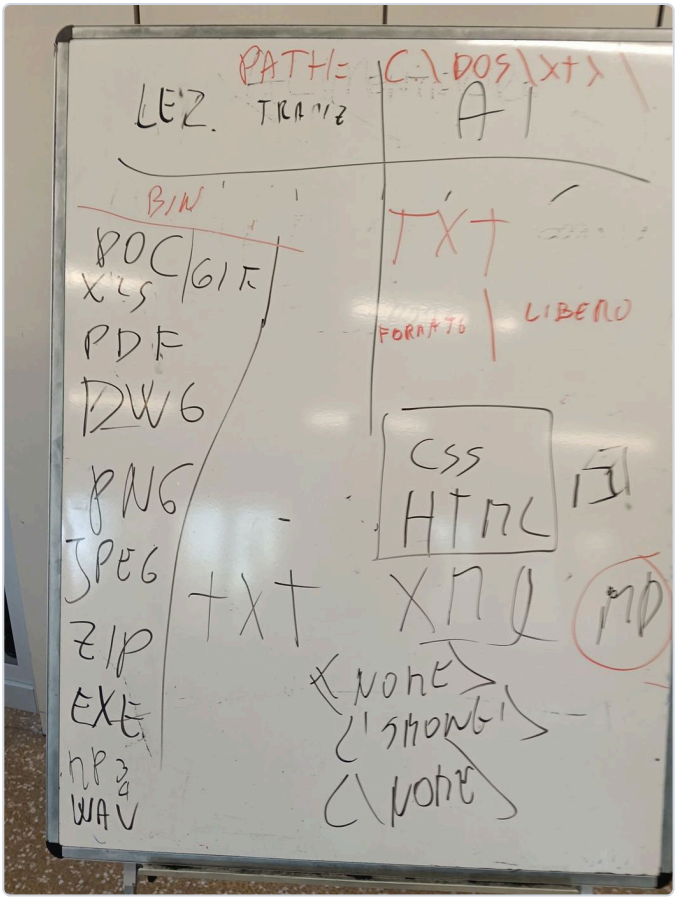
1. Lunedì 28/09/2026, 3 ore, Laboratorio.

03.2 Che cosa abbiamo fatto

1. La dispensa «Git e GitHub raccontati con parole semplici»: il problema, la storia di Git, le parole, il lavoro in due, il versioning e i grafi.
2. I tipi di file: estensione, testo e binario, file a tag, compressione, schermo e video; compito su Classroom.

03.3 La teoria

1. **Il problema che conosci già:** `ricerca.doc`, `ricerca_v2.doc`, `ricerca_DEFINITIVA.doc`, `ricerca_DEFINITIVA_vera.doc`… Non sai più qual è l'ultima buona, non torni indietro se cancelli una frase, e in due ci si cancella il lavoro. Git nasce per risolvere questo.
2. **Git è una macchina del tempo:** ogni volta che vuoi gli dici «salva com'è adesso»; Git scatta una fotografia e la mette in fila con le altre, tutte in una sola cartella. Puoi tornare a qualsiasi fotografia del passato.
3. **Un po' di storia:** nel 2005 **Linus Torvalds**, il creatore di **Linux**, aveva migliaia di programmatori che lavoravano insieme sullo stesso codice. Si scrisse da solo, in pochi giorni, uno strumento velocissimo che non perde mai i dati: Git. Oggi è lo standard nel mondo del lavoro.
4. **Git e GitHub:** Git è lo strumento, gratis e **open source** (il suo codice è pubblico); GitHub è il sito dove metti i progetti online. Git è il motore, GitHub il garage. Non è l'unico garage: ci sono anche GitLab e Bitbucket.
5. **PaaS, Freemium e la famiglia «…aaS»: IaaS** ti dà i server nudi (affitti il locale vuoto); **PaaS** la piattaforma pronta, come GitHub (affitti la cucina attrezzata); **SaaS** il programma pronto nel browser, come Gmail (ti portano la pizza pronta). **Freemium** = base gratis, funzioni in più a pagamento.
6. **Il lavoro in due, passo passo:**
   6.1 **clone**: la prima volta ognuno scarica il progetto sul suo computer;
   6.2 **lavora**: ognuno sul suo pezzo;
   6.3 **commit**: ogni tanto una fotografia del proprio lavoro, con un messaggio;
   6.4 **push**: manda le fotografie online, su GitHub;
   6.5 **pull**: scarica le novità dell'altro;
   6.6 **Pull Request**: «ho fatto questa cosa, la guardi? se ti va la uniamo». L'altro la legge e, se è d'accordo, fa il merge. Noi facciamo tutto dal browser, cliccando.
7. **Il versioning v1.2.3:** major (cambia tutto), minor (funzione nuova), patch (solo un bug). In pratica: fai i commit; quando il progetto è pronto e provato scegli il numero (si parte da v1.0.0); metti un **tag** (una bandierina) su quel commit; pubblichi una **Release**, che non cambia più; alla modifica dopo aumenti il numero.
8. **I tipi di file:**



La lavagna del 28/09: i file binari (doc, xls, pdf, dwg, png, jpeg, zip, exe, mp3, wav) e i file di testo a formato libero (txt, css, html, xml, md); il path C:\DOS

8.1 L'**estensione** (.mp4, .md…) è il cognome del file: dice il tipo e con quale programma si apre; il **percorso** (path) è il suo indirizzo, per esempio `C:\Documenti\appunti.txt`;
8.2 i file di **testo** si leggono con il Blocco note (txt, md, html, css, xml, csv, json); i file **binari** mostrano simboli strani e vogliono un programma (doc, xls, pdf, dwg, png, jpg, zip, exe, mp3, wav);
8.3 i file **a tag** (HTML, XML, MD) marcano il testo con etichette per dargli struttura: `<h1>` apre, `</h1>` chiude;
8.4 la **compressione** senza perdita torna identica (ZIP, PNG); con perdita è più leggera ma butta un po' di dettaglio (JPG, MP3, MP4);
8.5 **refresh rate** = quante volte al secondo si aggiorna lo **schermo** (in Hz); **fps** = fotogrammi al secondo di un **video**. Sono cose diverse ma collegate: un video a 60 fps si vede fluido solo su uno schermo che si aggiorna almeno 60 volte al secondo.

03.4 Da ricopiare sul quaderno

```
GIT = la macchina del tempo del progetto (lo strumento)
GITHUB = il sito dove il progetto sta online (il servizio)
clone -> commit -> push -> pull -> Pull Request -> merge
v1.2.3 = major . minor . patch
```

03.5 Esercizi e compiti

1. **I tipi di file**
   1.1 Testo del compito: obiettivo, dimostrare che hai capito che cosa sono i tipi di file (formati, tag, compressione, video) con parole tue. Puoi usare la dispensa per ripassare, ma le risposte

devono essere tue.
1.2 Apri un documento nuovo: nel browser vai su `docs.new` e dai un titolo, per esempio «I tipi di file» con il tuo nome. Rispondi alle parti.
1.3 Parte 1, le basi: che cos'è un file e a che cosa serve l'estensione (2-3 frasi); perché rinominare `foto.jpg` in `foto.txt` non la trasforma in testo.
1.4 Parte 2, tre formati diversi: scegli tre formati di famiglie diverse (uno di testo, uno immagine o video, uno archivio) e per ognuno scrivi a che cosa serve e con quale programma si apre.
1.5 Parte 3, un file a tag: scrivi un piccolo esempio di 3-4 righe (HTML, XML o MD) e spiega che cosa fanno i tag. Esempio di partenza: `<h1>Il mio titolo</h1>` e `<p>Un paragrafo con una parola in <b>grassetto</b>.</p>`.
1.6 Parte 4, compressione: la differenza tra senza perdita e con perdita, con un esempio per ciascuna (quale formato e in quale situazione).
1.7 Parte 5, schermo e video: con parole tue, che differenza c'è tra il refresh rate di un monitor (Hz) e gli fps di un video; perché sono due cose diverse ma collegate.
1.8 Parte 6, testo o binario e il percorso: come capisci se un file è di testo o binario (un esempio di ciascuno); il percorso di un file d'esempio, per esempio `C:\Documenti\nomefile.txt`.
1.9 Quando hai finito, consegna il documento su Classroom. Conta soprattutto saperlo spiegare a voce.
1.10 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/MjU0MTM4NjQ3MDda/details
1.11 Che cosa si consegna: il Documento con tutte le parti.

04 Mercoledì 30/09/2026 — Una pagina web da zero

04.1 Quando

1. Mercoledì 30/09/2026, 3 ore, Laboratorio.

04.2 Che cosa abbiamo fatto

1. La prima pagina HTML sul proprio PC, con il Blocco note: struttura, tag, colori; quattro mini-compiti; poi lo stile spostato in un file CSS separato e la sfida finale.

04.3 La teoria

1. **HTML = contenuto, CSS = aspetto.** La pagina sta nella cartella `mio-sito` con due file: `index.html` (i testi) e `style.css` (i colori). Doppio clic su index.html e la pagina si apre nel browser.
2. **La struttura della pagina:**
   2.1 `<head> … </head>`: istruzioni per il browser, non si vedono (il titolo della scheda, il collegamento al CSS);
   2.2 `<link rel="stylesheet" href="style.css">`: collega l'HTML al file style.css;
   2.3 `<body> … </body>`: tutto quello che si vede;
   2.4 `<h1>` titolo grande, `<h2>` titolo di sezione, `<p>` paragrafo, `<ul>` elenco con un `<li>` per ogni voce, `<div class="scheda">` una scatola a cui il CSS dà l'aspetto.
3. **Una regola CSS:** `body { background: lightblue; }`. Prima il **selettore** (a chi si applica: `body`, `h1`, `.scheda` con il punto per una classe), poi tra le graffe le **proprietà** (`background`, `color`, `font-family`, `padding`, `border-radius`), ognuna chiusa da `;`.

```
<!DOCTYPE html>
<html lang="it">
<head>
  <meta charset="utf-8">
  <title>Il mio primo sito</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Ciao, sono Leo</h1>
  <div class="scheda">
    <h2>Le mie passioni</h2>
    <ul>
      <li>Calcio</li>
      <li>Musica</li>
    </ul>
  </div>
</body>
</html>
```

4. **Salvare con il Blocco note:** File, Salva con nome, cartella `mio-sito`, «Salva come: Tutti i file», nome `index.html`, codifica **UTF-8**. Nomi tutti minuscoli, senza spazi, senza `.txt` alla fine.
5. **Vedere un cambiamento:** salva con `Ctrl + S`, poi nel browser `F5`.

6. **Il CSS in un file separato:** prima lo stile stava nell'HTML, dentro `<style> … </style>`; si copiano le regole in un file nuovo (`stile.css` o `style.css`), si cancella il blocco `<style>` e al suo posto si scrive `<link rel="stylesheet" href="stile.css">`. I due file devono stare nella **stessa cartella**, altrimenti la pagina non trova lo stile.
7. **La sfida finale:** un sottotitolo `<h2>`, una lista `<ul><li>`, un link `<a href="…">`, un'immagine `<img src="…" width="200">`, un carattere diverso (`font-family: Verdana;`).

04.4 Da ricopiare sul quaderno

1. La cartella `mio-sito` con dentro i due file, una freccia da index.html a style.css con scritto `<link>`, e sotto: **HTML = contenuto, CSS = aspetto**.

04.5 Esercizi e compiti

1. **Esercitazione pagina HTML**
   1.1 Testo (il foglio «La mia pagina web, 4 mini-compiti in fila»): un passo alla volta, ognuno finisce con una vittoria.
   1.2 Mini-compito 1, la pagina che si apre: apri il Blocco note; copia tutto il codice della dispensa; salva con File, Salva con nome, Tutti i file, nome `presentazione.html` sul Desktop; doppio clic sul file. Fatto quando nel browser vedi la pagina con scritto «Mi presento».
   1.3 Mini-compito 2, la pagina sei tu: nel Blocco note cambia i dati di esempio con i tuoi; salva (Ctrl+S) e nel browser premi F5. Fatto quando la pagina mostra il tuo nome e i tuoi dati.
   1.4 Mini-compito 3, cambia i colori: trova `lightblue` e cambialo (per esempio `pink`, `lightgreen`, `gold`); trova `darkblue` (il colore del titolo) e cambialo; salva e premi F5 ogni volta. Sfida: prova 3 colori e tieni la combinazione più bella.
   1.5 Mini-compito 4, rendila speciale e mostrala: aggiungi una riga tua nella scheda (un hobby, una frase su di te); chi vuole aggiunge una foto (`foto.jpg` nella stessa cartella e `<img src='foto.jpg' width='150'>`); fai vedere la pagina al compagno.
   1.6 Consegna: crea un Documento Google e incolla dentro uno screenshot della tua pagina e il codice copiato dal Blocco note; scrivi la riflessione (che cosa non sono riuscito a fare e perché, che cosa non ho capito); consegna il Documento su Classroom.
   1.7 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/MjU0MjU4ODM3MTRa/details
   1.8 Che cosa si consegna: il Documento con screenshot, codice e riflessione.
2. **CSS in un file separato**
   2.1 Testo (il foglio «Sposta il CSS in un file separato»): finora HTML e stile stanno in un solo file; ora li separiamo, il testo resta nell'HTML, i colori vanno in `stile.css`.
   2.2 Trova il blocco stile: nel file HTML, dentro `<head>`, trova `<style> … </style>`.
   2.3 Crea il file stile.css: apri un nuovo Blocco note e copia solo le regole (senza i tag `<style>`), per esempio `body { background: lightblue; font-family: Arial; text-align: center; }` e `h1 { color: darkblue; }`; salva con Tutti i file, nome `stile.css`, nella stessa cartella di presentazione.html.
   2.4 Collega il CSS all'HTML: nel file HTML cancella tutto il blocco `<style> … </style>` e al suo posto, dentro `<head>`, scrivi `<link rel="stylesheet" href="stile.css">`.
   2.5 Salva e guarda: salva l'HTML e premi F5; la pagina è identica, ma lo stile arriva dal file esterno.
   2.6 Prova che funziona: apri stile.css, cambia un colore, salva e premi F5: il colore cambia.
   2.7 Consegna: aggiorna il tuo Documento con uno screenshot della pagina, il codice HTML, il codice di stile.css e la riflessione.
   2.8 Sfida finale (stesso Documento): aggiungi un `<h2>`, una lista, un link, un'immagine dal web e un carattere diverso; nuovo screenshot, codice aggiornato e che cosa hai aggiunto.

2.9 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/MTYyNzM3MDI0ODgz/details
2.10 Che cosa si consegna: il Documento aggiornato.

05 Venerdì 02/10/2026 — La pagina «Mi presento» con il CSS separato

05.1 Quando

1. Venerdì 02/10/2026, Laboratorio.

05.2 Che cosa abbiamo fatto

1. La pagina «Mi presento» rifatta in 7 passi, con il CSS in un file separato: la cartella `mio-sito` con `index.html` e `style.css`, pronta per andare su Internet.

05.3 La teoria

1. **I 7 passi:** una cartella `mio-sito` sul Desktop; il codice del riquadro 1 copiato con il bottone «Copia»; incollato nel Blocco note; il tuo nome al posto di «Leo» (niente cognome); salvato come `index.html` con «Tutti i file»; un foglio nuovo con il riquadro 2 salvato come `style.css`; doppio clic su index.html.
2. **Il collegamento:** è la riga `<link rel="stylesheet" href="style.css">` a far usare all'HTML i colori del CSS. Se il nome del file non è uguale lettera per lettera, la pagina resta senza colori.
3. **Bonus:** apri style.css con il Blocco note, cambia `lightblue` con `pink` o `gold`, salva e premi F5.

05.4 Esercizi e compiti

1. **Pagina HTML**
   1.1 Il compito di Classroom con la pagina di questa lezione; il suo testo non è conservato nell'archivio del corso. Il lavoro è «La mia pagina web in 7 passi» (punto 1 della teoria): alla fine una pagina web con il tuo nome che si apre nel browser, in circa 20 minuti.
   1.2 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/MjU0MjU2ODQyNDZa/details
   1.3 Che cosa si consegna: la pagina «Mi presento» con i due file, index.html e style.css.

06 Lunedì 05/10/2026 — Il sito pubblico su GitHub Pages

06.1 Quando

1. Lunedì 05/10/2026, Laboratorio.

06.2 Che cosa abbiamo fatto

1. La Dispensa 2: il sito personale pubblicato su Internet con GitHub Pages, tutto dal browser; il compito «Il mio primo sito pubblico».

06.3 La teoria

1. **Che cosa ottieni:** un indirizzo Internet vero, come `https://tuonome.github.io/mio-sito/`, che chiunque può aprire, anche sul telefono.
2. **Le parole nuove: repository** = la cartella del progetto su GitHub; **commit** = salvare una versione; **upload** = caricare; **GitHub Pages** = il servizio gratis che trasforma il repository in un sito.
3. **Controllo privacy prima di caricare:** niente cognome, telefono, email, indirizzo, foto del viso. Quello che carichi lo vede il mondo intero.
4. **I passi:**
   4.1 su github.com, «Sign in» con nome utente e password;
   4.2 in alto a destra il **+**, «New repository»; nome `mio-sito`, pubblico, «Create repository»;
   4.3 «Add file» (il bottone grigio a sinistra del verde «Code»), «Upload files»; trascina `index.html` e `style.css`;
   4.4 il bottone verde «Commit changes»: hai fatto un commit, una versione salvata;
   4.5 «Settings» (l'ingranaggio), a sinistra «Pages», Branch `main` e `/ (root)`, «Save»; dopo un minuto in alto compare l'indirizzo del sito.
5. **Se la pagina di GitHub è tradotta** i nomi dei bottoni cambiano: cerca il bottone per posizione e colore.
6. **Cambiare il sito dopo:** nel repository clic sul file, la **matita** in alto a destra, cambia, bottone verde «Commit changes». Dopo un minuto, F5 sul sito. Ogni modifica è una nuova versione.

06.4 Da ricopiare sul quaderno

1. Il tuo indirizzo `github.io` e lo schema: **PC** (cartella mio-sito) → **upload + commit** → **GitHub** (repository) → **Pages** → **sito pubblico**.

06.5 Esercizi e compiti

1. **Pagina personale HTML**
   1.1 Testo (il foglio «Compito: il mio primo sito pubblico»): consegni il Documento già pronto nel compito su Classroom (c'è già il tuo nome), con 5 parti separate. HTML e CSS non vanno insieme: ognuno nella sua parte.
   1.2 Apri il Documento: su Classroom apri il compito e clicca sul Documento con il tuo nome.
   1.3 Parte 1, il link: apri il tuo sito pubblicato, clic sulla barra dell'indirizzo, Ctrl + C; nel Documento sotto «1. Il link», Ctrl + V. Deve contenere `github.io`.
   1.4 Parte 2, tutto l'HTML: clic destro su index.html, «Apri con», «Blocco note»; Ctrl + A, Ctrl + C; nel Documento sotto «2. Codice HTML», Ctrl + V.
   1.5 Parte 3, tutto il CSS, a parte: la stessa cosa con style.css, sotto «3. Codice CSS». Non mischiarlo con l'HTML.

1.6 Parte 4, lo screenshot: apri il sito pubblicato; Win + Shift + S e seleziona la finestra con anche la barra dell'indirizzo; nel Documento sotto «4. Screenshot», Ctrl + V.
1.7 Parte 5, spiegalo con parole tue, 1-2 frasi: a che cosa serve il file style.css; che cosa non sono riuscito a fare e perché; che cosa non ho capito.
1.8 Consegna: torna sul compito in Classroom e clicca il bottone blu «Consegna».
1.9 Controlla prima di consegnare: il link si apre e contiene github.io; l'HTML è tutto, dalla prima riga `<!DOCTYPE html>` all'ultima `</html>`; il CSS è in una parte separata; lo screenshot mostra la barra dell'indirizzo; sul sito niente cognome, telefono, email; hai risposto alle 3 domande.
1.10 Valutazione (in decimi): sito pubblicato e link funzionante 2; HTML completo e corretto 2; CSS in un file separato e collegato 2; il sito è tuo 1,5; screenshot con la barra dell'indirizzo 1; spiegazione con parole tue 1; privacy rispettata 0,5. Non hai finito in tempo? Consegna lo stesso quello che hai fatto e scrivilo nella parte 5.
1.11 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/MTYyNzQ4MjgxODQz/details
1.12 Che cosa si consegna: il Documento con le 5 parti.

07 Mercoledì 07/10/2026 — Il sito migliorato: commit e History

07.1 Quando

1. Mercoledì 07/10/2026, Laboratorio.

07.2 Che cosa abbiamo fatto

1. La Dispensa 3: il sito migliorato direttamente su GitHub, con la matita: due bottoni-link che cambiano colore, l'ombra della scheda e la storia delle versioni (History).

07.3 La teoria

1. **Modificare su GitHub:** apri il file, la **matita** in alto a destra, scrivi, poi «Commit changes…» e di nuovo «Commit changes». Ogni modifica è un **commit**, una nuova versione salvata nella storia.
2. **I pezzi nuovi:**
   2.1 `<a href="…">testo</a>`: un link; **href** è l'indirizzo dove porta;
   2.2 `target="_blank"`: apre il link in una scheda nuova;
   2.3 `class="bottone"`: il nome che usa il CSS per dare l'aspetto;
   2.4 `.bottone:hover`: lo stile quando il mouse passa sopra;
   2.5 `box-shadow`: l'ombra della scatola.

```
.bottone {
  display: inline-block;
  background: darkblue;
  color: white;
  padding: 8px 14px;
  border-radius: 8px;
  text-decoration: none;
}
.bottone:hover {
  background: orange;
}
```

3. **La History:** nel repository clic sul file, in alto a destra «History» (l'icona con l'orologio): la lista dei commit, ognuno con data e messaggio. Se il sito si rompe, guardi la versione di prima e la ricopi: con Git niente si perde.
4. **Non vedi i cambiamenti?** Aspetta un minuto e premi `Ctrl + F5` (ricarica forzata).

07.4 Da ricopiare sul quaderno

1. Una linea del tempo con i tuoi commit (un pallino per ogni versione) e sotto che cosa hai cambiato in ognuno.

07.5 Esercizi e compiti

1. **Compito 3 — Il sito migliorato**
   1.1 Testo del compito: oggi miglioriamo il sito direttamente su GitHub: bottoni-link, ombra e la storia delle versioni (commit).

1.2 Tutto il materiale (dispensa 3, codice con il bottone Copia, compito) è qui: https://nicolaregge-pulse.github.io/corso-informatica/3inf-sito/
1.3 Compila il Documento del compito (c'è già il tuo nome) e premi «Consegna». Il sito è pubblico: solo nome o soprannome.
1.4 Le parti del Documento: parte 1, il link del tuo sito; parte 2, lo screenshot del sito con i due bottoni-link; parte 3, il codice CSS che hai aggiunto (con i tuoi colori); parte 4, lo screenshot della History (la lista dei commit); parte 5, spiegalo con parole tue: a che cosa serve href, che cos'è un commit e perché con Git niente si perde; poi la riflessione (che cosa non sei riuscito a fare, che cosa non hai capito) e il bottone blu «Consegna».
1.5 Valutazione (in decimi): sito con i due bottoni funzionanti 3; CSS aggiunto e personalizzato 2; screenshot della History 2; spiegazione con parole tue 2; riflessione 1.
1.6 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/ODg5NTM0NjQ5MDYw/details
1.7 Che cosa si consegna: il Documento con le 5 parti e la riflessione.

08 Giovedì 08/10/2026 — Il treno blindato: squadre e Pull Request

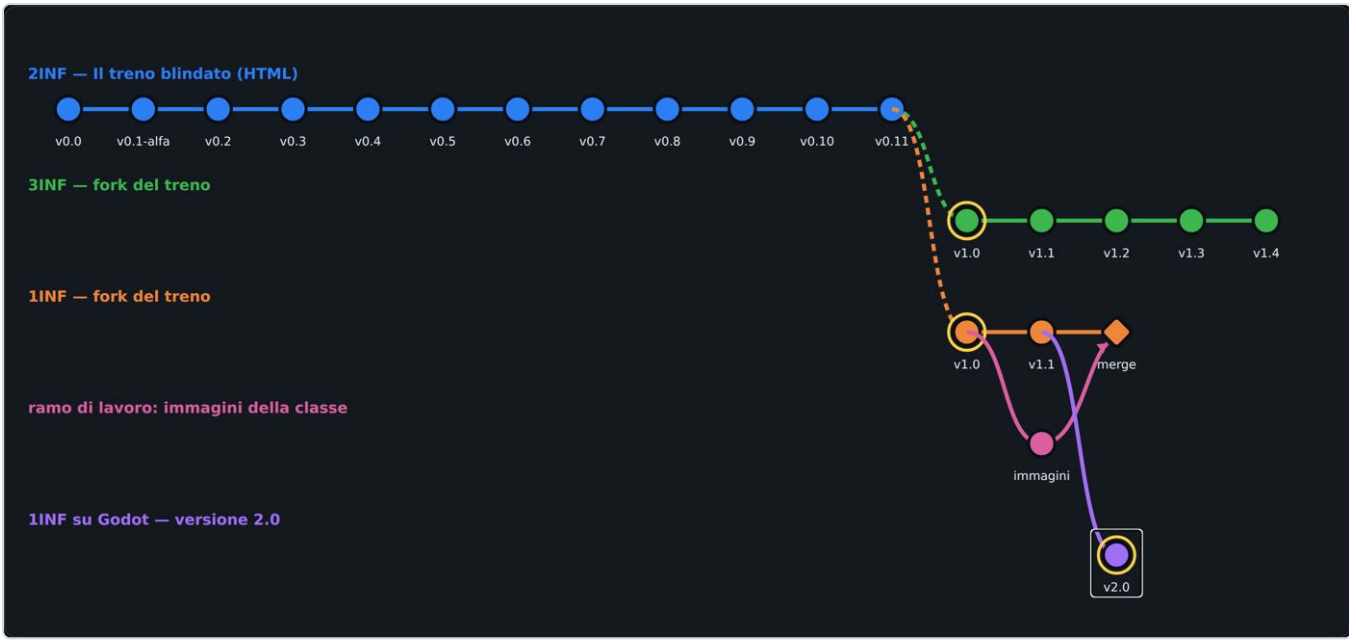
08.1 Quando

1. Giovedì 08/10/2026, 2 ore, Laboratorio.

08.2 Che cosa abbiamo fatto

1. La classe diventa un'azienda di software: parte dal gioco della 2INF «Il treno blindato» e ne fa la sua copia (**fork**) per migliorarla.
2. Ogni consegna su Classroom è una **Pull Request**: gli sviluppatori (il prof con l'AI) fanno il merge e pubblicano le versioni nuove (dalla v1.0 alla v1.4 della 3INF).
3. Il grafo delle versioni, letto e disegnato a mano.

08.3 La teoria



Il grafo delle versioni del Treno blindato: il ramo della 2INF, i fork della 3INF e della 1INF, il ramo delle immagini e la v2.0 su Godot

1. **Perché i numeri cambiano così:** v0.x = le prove, prima della versione stabile (la 2INF ha fatto 12 prove, dalla v0.0 alla v0.11); 1.0 = la prima versione vera di un ramo (il fork della 3INF e quello della 1INF); 1.1, 1.2 = versioni **minor**, piccole aggiunte (un cattivo, un cielo); 2.0 = versione **major**, cambia una cosa grossa (il motore Godot): il primo numero sale e il secondo riparte da 0.
2. **Perché si fanno i rami:** per provare una cosa nuova senza rompere il gioco che funziona. Nelle aziende si lavora sempre così: ognuno sul suo ramo, poi Pull Request, controllo e merge.
3. **Perché Godot (v2.0):** Godot è un motore vero per videogiochi, gratis e portabile, usato anche nel lavoro. I cattivi ci sono ancora: stanno nei file di dati, uguali a prima. Lazarus **reagisce** (aspetta un clic), Godot **pulsa** (gira da solo circa 60 volte al secondo).
4. **La storia del fork della 3INF:** v1.0 traversine di legno e nuvole; v1.1 un cattivo nuovo con la mimetica nera e argento (Pull Request #1 e #2 di SCENARI); v1.2 si parte dalla montagna e il

tempo peggiora a ogni livello; v1.3 cespugli, greggi, armi diverse, nuovi cattivi; v1.4 treno più veloce, nuovi personaggi, il difetto del cielo sistemato.

5. **Le parti di una Pull Request:** che cosa cambia, dove, perché; si possono allegare disegni, foto, screenshot, immagini fatte con l'IA.
6. **Le fasi della giornata, come uno sprint:** PR-1 il tuo cattivo; PR-2 una modifica per la tua squadra; PR-3 un difetto trovato nella versione nuova; PR-4 la revisione della Pull Request di un compagno; alla fine «che cosa ho imparato».

08.4 Da ricopiare sul quaderno

1. Le parole del grafo (versione, ramo, fork, merge, Pull Request, changelog, major, minor), ognuna con la traduzione nella tua lingua.
2. Un grafo a mano con 3 colori: il ramo principale, un fork e un ramo di lavoro che rientra con un merge; sotto ogni pallino il numero di versione.

08.5 Esercizi e compiti

1. **Compito 4 — Il treno blindato della 3INF: le mie Pull Request**
   1.1 Testo del compito: oggi siete un'azienda di software. Partite dal gioco della 2INF, «Il treno blindato»: la 3INF ne fa la sua copia (un fork) e la migliora.
   1.2 Si consegna solo su Classroom, nel Documento del compito: ogni punto è una Pull Request (proposta di modifica). Gli sviluppatori (il prof con l'IA) leggono ogni 5 minuti, fanno il merge e pubblicano la versione nuova con la vostra squadra e il numero del PC. Potete allegare file: disegni, foto, screenshot, immagini fatte con l'IA («Aggiungi o crea», «File»).
   1.3 Subito: punto 0 del Documento (squadra, compagni, capo squadra, ruolo, PC).
   1.4 Tutti consegnano tutte le Pull Request, ognuno nel suo Documento: PR-1 il tuo cattivo; PR-2 una modifica per la tua squadra; PR-3 un difetto nella versione nuova; PR-4 la revisione della Pull Request di un compagno; punto 5, che cosa ho imparato, poi premi «Consegna».
   1.5 Voto: ogni Pull Request accettata (merge) vale punti per te e per la tua squadra. Chi non consegna una fase entro l'orario perde i punti di quella fase.
   1.6 Pull Request, merge, versioni e report in tempo reale: https://nicolaregge-pulse.github.io/corso-informatica/3inf-gioco/
   1.7 Il gioco della 3INF: https://nicolaregge-pulse.github.io/corso-informatica/giochi/treno-3inf/
   1.8 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/MjU0NTYwNzQ3OTRa/details
   1.9 Che cosa si consegna: il Documento con le PR-1, PR-2, PR-3, PR-4 e che cosa ho imparato.

09 Lunedì 12/10/2026 — Versioning e Agile

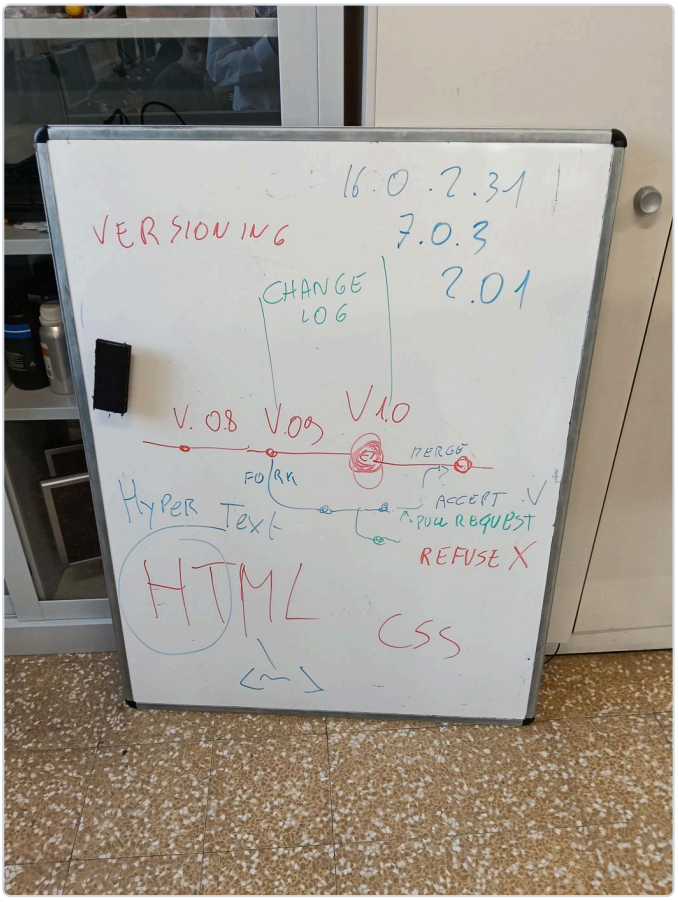
09.1 Quando

1. Lunedì 12/10/2026, 3 ore, Laboratorio.

09.2 Che cosa abbiamo fatto

1. Il proprio Documento come ramo dell'albero delle versioni; come si chiede una modifica (Pull Request PR1, PR2…); il Manifesto Agile e le 10 regole; il backlog Agile; il feedback sulla teoria.

09.3 La teoria



La lavagna del versioning: numeri di versione, il ramo principale con v0.8, v0.9 e v1.0, il changelog, il fork, il merge, la Pull Request accettata o rifiutata; HTML e CSS

1. **Le 10 parole, con il nostro gioco:**
   1.1 **Versione**: una fotografia del progetto in un momento preciso, con un numero;
   1.2 **Git e GitHub**: Git ricorda tutta la storia del progetto (chi, che cosa, quando, perché); GitHub è il sito dove il repository sta online;
   1.3 **Commit**: un salvataggio nella storia, con un messaggio; il codice accanto (per esempio 7724900) è il suo numero;
   1.4 **Branch** (ramo): una linea di lavoro separata, per provare senza rompere il gioco che funziona;
   1.5 **Fork** (copia): la copia di un progetto intero che cresce per conto suo; la 3INF ha fatto il fork del treno della 2INF alla v0.11;
   1.6 **Push**: mandare i propri commit dal PC al repository online;
   1.7 **Pull**: prendere da GitHub l'ultima versione e portarla sul proprio PC;

1.8 **Pull Request**: «ho fatto una modifica, la vuoi nel progetto?»; qualcuno la controlla e fa il **merge**, oppure la rifiuta;
1.9 **Changelog**: la lista, versione per versione, di che cosa è cambiato e perché;
1.10 **Numeri delle versioni**: v0.x prove; 1.0 prima versione stabile; 1.1, 1.2 minor; 2.0 major.

2. **La Pull Request ben scritta: Titolo**, **Cosa cambia**, **Dove**, **Perché**.
3. **Agile:** si lavora a piccoli passi, si consegna spesso qualcosa che funziona, si ascolta chi userà il programma e si corregge subito. Il **Manifesto Agile** dice per esempio: un programma che funziona conta più di tanti documenti; rispondere ai cambiamenti conta più che seguire un piano fisso; le persone e il parlarsi contano più degli strumenti.
4. **Il backlog:** la coda dei lavori, in ordine. Ogni richiesta ha un **peso** con i numeri di Fibonacci (1, 2, 3, 5, 8): 1 un colore, 2 un numero nelle regole, 3 un personaggio, 5 un ambiente, 8 due cose insieme o una regola nuova. Si fanno prima le più leggere; a parità di peso, chi è arrivato prima.

09.4 Da ricopiare sul quaderno

1. Le 10 parole del punto 1, ognuna con una frase tua.
2. La Pull Request in 4 parti: Titolo, Cosa cambia, Dove, Perché.

09.5 Esercizi e compiti

1. **Versioning: il tuo Documento nell'albero delle versioni**
1.1 Testo del compito: tutto in una sola pagina (il link resta sempre lo stesso, si aggiorna da solo): https://nicolaregge-pulse.github.io/corso-informatica/compito/3inf-20261012-documento/
1.2 Apri il link.
1.3 Fase 1: leggiamo insieme la teoria.
1.4 Fase 2: il lavoro arriva in fondo alla stessa pagina, quando il prof lo pubblica (il feedback sulla teoria e le richieste PR1, PR2…).
1.5 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/MTYyNzYxNzQzMTQw/details
1.6 Che cosa si consegna: il lavoro della pagina, fase 1 e fase 2.

10 Mercoledì 14/10/2026 — Il merge e la verifica sul versioning

10.1 Quando

1. Mercoledì 14/10/2026, Laboratorio.

10.2 Che cosa abbiamo fatto

1. Il merge dal vivo: le richieste della classe entrano nel gioco (backlog e sprint); ripasso, verifica su Moduli, valutazione e recupero.

10.3 La teoria

1. **Le versioni e il changelog:** il numero ha due parti, **major.minor**. Da v1.4 a v1.5 è un passo piccolo, minor; da v1.5 a v2.0 un cambio grande, major. Il changelog dice che cosa è cambiato e da quale richiesta. Un programma cambia perché chi lo usa chiede cose nuove e perché si trovano **difetti** da sistemare.
2. **Fork, ramo e merge:** il fork è una copia del progetto per lavorarci senza toccare l'originale; il ramo è una strada di lavoro parallela nello stesso progetto; il merge unisce una modifica dentro il gioco, e lo fa il prof quando accetta una Pull Request.
3. **La Pull Request** è una richiesta di modifica scritta, in quattro parti: Titolo, Cosa cambia, Dove, Perché.
4. **Il backlog e i pesi** di Fibonacci: 1, 2, 3, 5, 8 (ogni numero è la somma dei due prima).
5. **Agile: sprint e retrospettiva.** Si lavora a **sprint**, giri di lavoro brevi con un obiettivo, alla fine dei quali esce una versione che funziona; alla fine dello sprint c'è la **retrospettiva**: la squadra dice che cosa è andato bene e che cosa migliorare.
6. **Le regole della squadra:** il gioco è di tutta la classe, quindi **aggiungere sì, cancellare no** (se hai un'idea aggiungi un personaggio tuo, non cancelli quello di un compagno). Il gioco è pubblico: **niente foto di persone vere** (privacy); si usa un personaggio disegnato o di un videogioco.

10.4 Esercizi e compiti

1. **Il merge: le vostre richieste entrano nel gioco**
   1.1 Testo del compito: tutto in una sola pagina: https://nicolaregge-pulse.github.io/corso-informatica/compito/3inf-20261014-merge/
   1.2 Apri il link.
   1.3 Fai i passi dall'alto in basso. Il passo fatto diventa verde.
   1.4 Oggi: almeno 3 richieste (PR1, PR2, PR3).
   1.5 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/MTYyNzY3MzMxOTUw/details
   1.6 Che cosa si consegna: almeno 3 richieste.
2. **Ripasso e verifica: versioni, Pull Request, Agile**
   2.1 Testo del compito: tutto in una sola pagina: https://nicolaregge-pulse.github.io/corso-informatica/compito/3inf-20261014-verifica/
   2.2 Apri il link.
   2.3 Fai i passi dall'alto in basso: nome, ripasso, verifica.
   2.4 Valutazione e recupero compaiono nella stessa pagina.
   2.5 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/MTYyNzY3NjYxMTI4/details
   2.6 Che cosa si consegna: il ripasso e la verifica sulla pagina.

3. **VERIFICA: clicca qui**
   3.1 Clicca il link: https://docs.google.com/forms/d/e/1FAIpQLSchGpd-Ry8iniOyGfTks9Qhn4b5gnHG2C_nflJ1Ksw9u7mvOQ/viewform
   3.2 Una domanda per pagina.
   3.3 Alla fine premi «Invia». Fatto.
   3.4 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/MTYyNzY3NzY3MzQ2/details
   3.5 Che cosa si consegna: la verifica su Moduli inviata.

11 Giovedì 15/10/2026 — Sei il Product Manager: un team Agile

11.1 Quando

1. Giovedì 15/10/2026, 2 ore, Laboratorio.

11.2 Che cosa abbiamo fatto

1. Il gioco del Product Manager: due clienti, due sprint, priorità e backlog, le release v3.0 e v3.1; le cerimonie Agile; tre domande con parole proprie.

11.3 La teoria

1. **Perché Agile:** il rischio è lavorare tanto nella direzione sbagliata e accorgersene tardi. Si lavora a piccoli passi, si consegna spesso, si ascolta chi userà il programma. Meglio sbagliare presto su una cosa piccola che tardi su una grande.
2. **I ruoli:**
   2.1 **Product Manager**: parla con i clienti, capisce che cosa vale di più, decide l'ordine del backlog e che cosa entra nello sprint; sa dire anche dei «no»;
   2.2 **sviluppatori**: stimano il peso e scrivono il codice;
   2.3 **tester**: provano la versione prima della consegna e trovano i bug;
   2.4 **clienti**: pagano il lavoro e dicono che cosa serve; non tutti pesano uguale.
3. **Peso, rischio e valore:** il **peso** è quanto lavoro serve (1, 2, 3, 5, 8); il **rischio** è quanto la modifica può rompere il resto (1 basso, 2 medio, 3 alto); il **valore** è quanto serve ai clienti.
4. **I clienti non pesano uguale:** TrenoCity (cliente grande) conta × 3, la Ludoteca (cliente piccolo) conta × 1. Valore = (valore per TrenoCity × 3) + (valore per la Ludoteca × 1).
5. **La priorità: Priorità = Valore ÷ (Peso + Rischio)**. Prima si fanno quelle che valgono tanto e costano poco.
6. **Lo sprint:** l'unità di tempo del metodo Agile (in azienda una settimana, da lunedì a venerdì; nel gioco 15 minuti), con una capacità fissa: al massimo **8 punti** di lavoro. Alla fine si consegna una versione nuova.
7. **Le quattro cerimonie: Planning** (lunedì: si sceglie che cosa fare); **Daily** (ogni giorno, 15 minuti in piedi: che cosa ho fatto, che cosa faccio, che cosa mi blocca); **Review** (venerdì: si mostra la versione ai clienti); **Retrospettiva** (venerdì: il team decide come lavorare meglio).
8. **Il planning è mettere insieme i pezzetti:** scegliere e ordinare le richieste in modo che, dentro gli 8 punti, si ottenga il risultato più grande per i clienti.
9. **La release e il changelog:** alla fine dello sprint esce una release con un numero (v3.0, v3.1…); il changelog è il registro tecnico delle modifiche, le note di rilascio sono il racconto per chi usa il programma.
10. **Esempio sul treno:** correggere il bug dei nemici (peso 3, rischio 1, valore 18) ha priorità 4,5; la schermata GAME OVER (peso 2, rischio 1, valore 10) ha 3,3; il nuovo motore della fisica (peso 8, rischio 3, valore 14) ha 1,3. Con 8 punti prendi il bug (3) e il GAME OVER (2) più qualcosa di leggero.

11.4 Da ricopiare sul quaderno

```
PRIORITA' = VALORE / (PESO + RISCHIO)
SPRINT = capacita' 8 punti
Planning -> Daily -> Review -> Retrospettiva
```

11.5 Esercizi e compiti

1. **Sei il Product Manager: un team Agile al lavoro**
   1.1 Testo del compito: tutto in una sola pagina: https://nicolaregge-pulse.github.io/corso-informatica/compito/3inf-20261015-agile/
   1.2 Apri il link.
   1.3 Scrivi il tuo nome e inventa il nome della tua azienda.
   1.4 Leggi la teoria: chi fa che cosa in un team Agile.
   1.5 Gioca: sei il Product Manager, scegli le richieste per 2 sprint e consegna le versioni v3.0 e v3.1.
   1.6 Rispondi alle 3 domande con parole tue e premi il bottone verde «Consegna al prof».
   1.7 Ti blocchi? In fondo alla pagina c'è l'assistente del Prof Regge.
   1.8 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/MTYyNzY5ODgxMzI0/details
   1.9 Che cosa si consegna: le versioni v3.0 e v3.1 e le 3 risposte, con «Consegna al prof».

12 Lunedì 19/10/2026 — Il sito diventa portfolio

12.1 Quando

1. Lunedì 19/10/2026, 3 ore, Laboratorio.

12.2 Che cosa abbiamo fatto

1. Ripasso di HTML5, CSS3 e GitHub Pages; la pagina `competenze.html` «Cosa so fare» con il menu e il link nella Home; verifica (crocette, risposte scritte, indirizzo della pagina) e recupero; la sfida della foto di sfondo; i collaboratori in coppia per il lavoro del 21/10.

12.3 La teoria

1. **Dove siamo:** hai un sito pubblico in un repository GitHub (quasi tutti `mio-sito`), con almeno `index.html` e `style.css`. Chi ha fatto la Dispensa 4 ha anche una seconda pagina (`passione.html`) e un menu.
2. **Ripasso HTML:** `<h1>`, `<h2>`, `<p>`, `<ul>` con `<li>`, `<a href="pagina.html">Testo</a>` (un collegamento a un'altra pagina), `<div class="scheda">`; la pagina ha una testa `<head>` e un corpo `<body>`.
3. **Ripasso CSS:** una regola è selettore più proprietà tra graffe; il punto davanti (`.menu`) vuol dire «classe»; ogni proprietà finisce con `;`.
4. **Ripasso GitHub Pages:** per cambiare un file, la matita e «Commit changes» due volte; per un file nuovo «Add file», «Create new file», il nome, incolla, commit. Dopo il commit aspetta 1 minuto e premi `Ctrl+F5`. Se vedi **404**, il nome del file non è quello del link: controlla maiuscole e minuscole.
5. **Il portfolio:** la pagina che racconta che cosa sai fare, le competenze **tecniche** (HTML, CSS, GitHub, Lazarus, montare un PC…) e quelle **personali** (chiedo aiuto quando mi blocco, se sbaglio riprovo, aiuto un compagno). È il tuo biglietto da visita per lo stage: in 30 secondi un'azienda capisce chi sei.
6. **Regola d'oro (privacy):** la pagina è pubblica. Solo cose vere e positive, solo nome o soprannome: niente cognome, telefono, indirizzo, classe, scuola, foto di altri. Le cose personali vanno nella verifica, che legge solo il prof.
7. **Il menu** in tutte le pagine, subito dopo `<body>`:

```
<nav class="menu">
  <a href="index.html">Home</a>
  <a href="competenze.html">Cosa so fare</a>
</nav>
```

```
.menu {
  background: darkblue;
  padding: 10px;
  border-radius: 10px;
}
.menu a {
  color: white;
  text-decoration: none;
  font-weight: bold;
  margin: 0 12px;
}
.menu a:hover {
  color: orange;
}
```

8. **La seconda pagina (Dispensa 4):** `passione.html` usa lo stesso `style.css`: cambi un colore e cambia in tutto il sito. Se una pagina è senza colori, le manca la riga `<link rel="stylesheet" href="style.css">`.
9. **I collaboratori:** nel repository «Settings», «Collaborators», «Add people», il nome utente GitHub del compagno; lui accetta l'invito dalla campanella («Accept invitation»).
10. **La foto di sfondo** (per chi ha finito): carica `sfondo.jpg` e in `body { }` aggiungi `background-image: url("sfondo.jpg"); background-size: cover; background-attachment: fixed;`.

12.4 Da ricopiare sul quaderno

1. La mappa del sito: i rettangoli delle pagine (index.html, competenze.html, passione.html) con le frecce del menu, e una freccia da ognuna verso style.css.

12.5 Esercizi e compiti

1. **Il mio sito diventa il mio portfolio**
   1.1 Testo del compito: tutto in una sola pagina: https://nicolaregge-pulse.github.io/corso-informatica/verifica/?b=3inf-portfolio
   1.2 Apri il link e scrivi nome e cognome.
   1.3 Leggi le schede 1-7: il ripasso di HTML, CSS e GitHub Pages e il codice della pagina nuova.
   1.4 Schede 8-10: la pratica, un passo alla volta. Il tuo sito avrà la pagina «Cosa so fare»: il tuo portfolio per lo stage.
   1.5 Rispondi alle domandine sotto le schede: così il monitor sa a che punto sei.
   1.6 Ultimi 10 minuti: scheda 11, in coppia vi aggiungete come collaboratori del sito (serve mercoledì).
   1.7 Quando la pagina è online, clicca «2. Verifica»: crocette e qualche risposta con parole tue. Nella domanda «incolla qui» metti l'indirizzo della tua pagina.
   1.8 Ti blocchi? In fondo alla pagina c'è la casella «Chiedi aiuto all'assistente del Prof Regge».
   1.9 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/MjU0ODQwMTE1MDFa/details
   1.10 Che cosa si consegna: la pagina competenze.html online e la verifica.

13 Mercoledì 21/10/2026 — Struttura del sito, CSS e lavoro in coppia

13.1 Quando

1. Mercoledì 21/10/2026, 2 ore, Laboratorio.

13.2 Che cosa abbiamo fatto

1. L'albero del sito e i nomi giusti, la cartella `img`, l'immagine di sfondo, l'animazione con transition e @keyframes, il numero di versione; il lavoro in team (collaboratore, ramo, Pull Request e merge sul sito del compagno); quiz su Moduli.

13.3 La teoria

1. **Il sito in ordine:** nella cartella principale del repository servono `index.html`, `style.css`, `competenze.html`, `passione.html` e la cartella `img` con le immagini.
2. **Le tre regole dei nomi:**
   2.1 **tutto minuscolo**: per GitHub `Index.html` e `index.html` sono due file diversi;
   2.2 **niente spazi**: `mia-foto.jpg` sì, `mia foto.jpg` no;
   2.3 **il collegamento uguale al nome, lettera per lettera**: se il file è `sfondo.jpeg`, nel CSS scrivi `sfondo.jpeg`.
   2.4 Per rinominare su GitHub: apri il file, matita, cambia il nome in alto, «Commit changes»; per spostarlo nella cartella img scrivi `img/` davanti al nome.
3. **L'immagine di sfondo:** in `body { }` le righe `background-image: url("img/sfondo.jpg"); background-size: cover; background-attachment: fixed;`. Se il testo non si legge, il contenuto va in un riquadro semitrasparente: `main { background: rgba(255,255,255,0.85); padding: 20px; border-radius: 12px; }`.
4. **Le animazioni:** la **transition** è un cambiamento morbido quando passi sopra con il mouse (`nav a { transition: transform 0.3s; }` e `nav a:hover { transform: scale(1.15); }`); **@keyframes** è un'animazione che parte da sola. Poche animazioni e leggere: un sito che si muove troppo stanca.

```
@keyframes entra {
  from { opacity: 0; transform: translateY(-20px); }
  to   { opacity: 1; transform: translateY(0); }
}
h1 { animation: entra 1s ease; }
```

5. **A coppie: ramo, Pull Request, merge.** Il compagno ti aggiunge come collaboratore; nel suo repository apri un file, matita, e in basso scegli «Create a new branch» (un ramo, la tua copia per provare); aggiungi una sezione «Il mio compagno di lavoro»; «Propose changes» e «Create pull request»; il compagno guarda le modifiche e preme «Merge pull request». Poi vi scambiate i ruoli.
6. **Il numero di versione** nel piè di pagina: `<footer>Versione 1.0</footer>`. Ogni Pull Request accettata che aggiunge qualcosa fa salire il secondo numero (1.0, 1.1, 1.2); il primo sale solo per un sito tutto nuovo (2.0).
7. **Il conflitto:** se due rami cambiano la stessa riga, GitHub dice «conflitto»: si sceglie insieme quale tenere («Resolve conflicts»), poi merge.

13.4 Da ricopiare sul quaderno

```
NOMI: tutto minuscolo, niente spazi, uguale lettera per lettera
ramo -> Pull Request -> merge
Versione 1.0 -> 1.1 (aggiunta) ... 2.0 (sito tutto nuovo)
```

13.5 Esercizi e compiti

1. **OGGI: il mio sito in ordine, più bello, e lavoro in coppia**
   1.1 Testo del compito: oggi fate solo queste 4 cose, in questo ordine.
   1.2 Leggi le 6 schede sulla pagina della lezione (scrivi nome e cognome): https://nicolaregge-pulse.github.io/corso-informatica/verifica/?b=3inf-grafica-pr
   1.3 Metti in ordine il tuo sito: apri «L'albero del mio sito», scrivi l'indirizzo del tuo sito, premi «Controlla» e sistema le croci rosse finché è tutto verde: https://nicolaregge-pulse.github.io/corso-informatica/3inf-albero/
   1.4 A coppie: sfondo, animazione, poi Pull Request sul sito del compagno, merge e numero di versione (schede 3-6).
   1.5 Fai il quiz (16 domande) e premi «Invia»: https://docs.google.com/forms/d/e/1FAIpQLSctBhCSlISxwcEd07VHtZ14scAfgqDYx3GXRV3L1CK962FSkQ/viewform
   1.6 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/ODkzNDcyNTczODAw/details
   1.7 Che cosa si consegna: il sito in ordine, la Pull Request in coppia e il quiz su Moduli.

14 Giovedì 22/10/2026 — Recupero: il sito di 3 pagine

14.1 Quando

1. Giovedì 22/10/2026, Laboratorio.

14.2 Che cosa abbiamo fatto

1. Lezione di recupero per tutti: ognuno arriva ad avere il suo sito personale che funziona (3 pagine con il menu, foto di sfondo, animazione, numero di versione), lavorando solo su github.com.

14.3 La teoria

1. **I file del sito,** tutti nella cartella principale (la foto nella cartella img):

```
index.html        la Home
style.css         i colori, lo sfondo, l'animazione
competenze.html   che cosa so fare
passione.html     la mia passione
img/sfondo.jpg    la foto di sfondo
```

2. **Come si mette un file su GitHub:** se il file non c'è, «Add file», «Create new file», il nome, incolla, «Commit changes…» e «Commit changes»; se c'è già ma va rifatto, aprilo, matita, `Ctrl+A`, incolla, commit.
3. **La cartella img:** «Add file», «Create new file», come nome `img/LEGGIMI.md` (la barra crea la cartella); poi nella cartella img «Upload files» con `sfondo.jpg`. La foto deve finire con `.jpg` (non .jfif o .webp).
4. **Accendere il sito:** «Settings», «Pages», Branch `main` e `/ (root)`, «Save»; in alto compare l'indirizzo `https://tuonome.github.io/nome-del-repository/`. Le modifiche arrivano dopo 1 o 2 minuti: `Ctrl+F5`.
5. **Se qualcosa non va:** 404 cliccando il menu, il nome del file è sbagliato; niente sfondo, la foto non si chiama sfondo.jpg o non è in img; il sito non cambia, aspetta e `Ctrl+F5`; tutto senza colori, manca style.css o ha un altro nome; file in una sottocartella, il sito li cerca nella cartella principale.

14.4 Esercizi e compiti

1. **OGGI: recupero — il mio sito personale funziona**
   1.1 Testo del compito: oggi fate solo questo, il vostro sito personale deve funzionare.
   1.2 Apri la lezione: https://nicolaregge-pulse.github.io/corso-informatica/3inf-recupero
   1.3 Guarda la tabella in alto: ti dice che cosa ti manca. Fai solo quei passi.
   1.4 I file pronti da copiare sono nel bottone verde «I FILE PRONTI DA COPIARE».
   1.5 Alla fine fai il quiz (linguetta «2. Verifica») e premi «Consegna».
   1.6 Bloccato? Assistente in fondo alla pagina.
   1.7 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/MTYyNzg5NDE3NjQx/details
   1.8 Che cosa si consegna: il sito di 3 pagine che funziona e il quiz.

15 Lunedì 26/10/2026 — Object Pascal e Java

15.1 Quando

1. Lunedì 26/10/2026, 3 ore, Laboratorio.

15.2 Che cosa abbiamo fatto

1. Lo stesso programma scritto in due linguaggi: Object Pascal (Lazarus) e Java. Hello World, variabili, una finestra con un bottone; compilato e interpretato; perché per cominciare si usa Lazarus.

15.3 La teoria

1. **Due linguaggi: Object Pascal** è il linguaggio di Lazarus (e di Delphi); **Java** è molto usato nelle aziende e nelle app Android.
2. **Come diventano un programma:** Object Pascal è **compilato** e diventa un file **.exe** che parte da solo su ogni PC Windows; Java è compilato in **bytecode** e poi **interpretato** dalla **Java Virtual Machine**, che deve essere presente sul PC.
3. **Le finestre:** in Lazarus le disegni con il mouse; in Java le scrivi con il codice.
4. **Hello World:**

```
begin
  writeln('Hello World');
end.
```

```
public class Ciao {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

5. **Perché Java è più lungo:** in Java già per scrivere una frase servono una **classe** e il metodo **main**, con parole come public, static, void, `String[] args`; in Object Pascal bastano begin ed end.
6. **Le variabili:** in Object Pascal si dichiarano nella sezione `var` con nome e tipo e si usano subito, anche un array (`voti: array[1..5] of integer;`). In Java i tipi semplici (int, double) vanno bene, ma per un array o per leggere dalla tastiera bisogna creare un oggetto con **new** (`int[] voti = new int[5];`). In Java l'array parte da 0.
7. **Una finestra con un bottone:** in Lazarus trascini il bottone, doppio clic, e scrivi una riga (`ShowMessage('Hello World');`); tutto il resto lo scrive Lazarus. In Java (con **Swing**) la finestra e il bottone si scrivono con il codice, ognuno creato con new (`JFrame`, `JButton`, `addActionListener`).
8. **Provare Java senza installare niente:** nel browser con **JavaFiddle** (`javafiddle.leaningtech.com`), se il firewall della scuola lo lascia passare; oppure Java portabile in versione .zip, scompattato in una cartella.
9. **Per essere onesti:** Java è potentissimo e chiesto nel lavoro; ma per imparare a programmare e vedere subito il risultato Object Pascal con Lazarus è la strada più semplice: meno parole da imparare, finestre con il mouse, un .exe che funziona ovunque.

10. **Prova tu:** Hello World è 1 riga in Object Pascal (dentro begin…end) e 5 in Java; per un array di 5 numeri in Java serve new; un programma Lazarus parte ovunque perché è compilato in un .exe, Java invece ha bisogno della sua macchina virtuale.

15.4 Da ricopiare sul quaderno

```
OBJECT PASCAL (Lazarus): compilato -> .exe, finestre con il mouse
JAVA: bytecode -> Java Virtual Machine, finestre con il codice (new)
Hello World: writeln('...')   /   System.out.println("...")
```

15.5 Esercizi e compiti

1. Nessun compito su Classroom con questa data: la pratica arriva il 28/10 (capitolo 16).

16 Mercoledì 28/10/2026 — Hello World in Lazarus e in Java

16.1 Quando

1. Mercoledì 28/10/2026, 2 ore (12-14), Laboratorio.

16.2 Che cosa abbiamo fatto

1. Hello World in Lazarus (progetto nuovo in `Documenti\Lazarus\Saluto`, TEdit, TButton, TLabel, ShowMessage) e in Java (nel browser con JavaFiddle e con **Eclipse** portabile: progetto HelloWorld, una finestra con un bottone); confronto riga per riga; consegna spostata alla lezione del 29/10.

16.3 La teoria

1. **Hello World** («ciao mondo») è il primo programma di ogni programmatore, in ogni linguaggio: serve a controllare che tutto funzioni.
2. **Partenza pulita in Lazarus:** File, «Chiudi tutto»; Progetto, «Nuovo progetto…», Applicazione, OK; subito «Salva progetto con nome» in `Documenti\Lazarus\Saluto`. Tutti i progetti stanno in Documenti, Lazarus: un progetto = una cartella, con i 4 file da tenere (`Saluto.lpi`, `Saluto.lpr`, `unit1.pas`, `unit1.lfm`).
3. **La finestra: TEdit** dove si scrive il nome (proprietà Text), **TButton** che si preme (Caption «Saluta»), **TLabel** dove compare il saluto (Caption).
4. **Il codice:**

```
procedure TForm1.Button1Click(Sender: TObject);
begin
  Label1.Caption := 'Ciao ' + Edit1.Text + '!';
  ShowMessage('Hello World, ' + Edit1.Text);
end;
```

5. **Le parti del codice:** `:=` assegna, «metti dentro»; `+` attacca due scritte; `'Ciao '` è una scritta fissa tra apici singoli; ShowMessage apre una finestrella con il bottone OK, la Label invece tiene il saluto nella finestra.
6. **Errori comuni in Lazarus:** manca il `;`, `=` invece di `:=`, manca un apice, la riga fuori da begin … end.
7. **Eclipse è un IDE** (ambiente di sviluppo): un programma che contiene l'**editor** (dove scrivi il codice), il **compilatore** (traduce e trova gli errori) e l'**esecuzione** (fa partire il programma). Anche Lazarus è un IDE: Eclipse è la stessa idea, per Java.
8. **Eclipse portabile:** lo ZIP si scompatta in una cartella corta (`C:\Users\Public\ec`), si apre `eclipse.exe`, si sceglie il **workspace** (la cartella dei progetti) con «Launch». Progetto nuovo «HelloWorld» (senza il file module-info.java); nella cartella `src` una classe `Main` con il metodo main; il bottone verde **Run**.
9. **La finestra in Java (Swing):**

```
import javax.swing.*;

public class Main {
    public static void main(String[] args) {
        JFrame finestra = new JFrame("Il mio primo programma");
        JButton bottone = new JButton("Premi qui");
        JLabel scritta = new JLabel("...");
        bottone.addActionListener(e -> scritta.setText("Hello World"));
        JPanel pannello = new JPanel();
        pannello.add(bottone);
        pannello.add(scritta);
        finestra.add(pannello);
        finestra.setSize(300, 120);
        finestra.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        finestra.setVisible(true);
    }
}
```

10. **Riga per riga, con Lazarus accanto:** `import javax.swing.*;` prende i pezzi per le finestre (in Lazarus `uses`); `public class Main` è il contenitore e ha lo stesso nome del file Main.java (in Lazarus la unit); `JFrame` è la finestra (il Form), `JButton` il bottone (TButton), `JLabel` la scritta (TLabel); `addActionListener` dice che cosa fare al clic (in Lazarus il doppio clic che crea Button1Click). Java distingue maiuscole e minuscole e chiude ogni blocco con le graffe `{ }`, Pascal con begin ... end.
11. **Errori comuni in Eclipse:** rosso sotto `system` o `jbutton` (maiuscole sbagliate); «Syntax error, insert ";"» (manca il punto e virgola); lo ZIP non scompattato tutto (percorso troppo lungo). L'errore è normale: Eclipse lo sottolinea in rosso e mette una X a sinistra della riga.

16.4 Da ricopiare sul quaderno

```
IDE = editor + compilatore + esecuzione (Lazarus, Eclipse)
Lazarus: Form, TButton, TLabel      Java: JFrame, JButton, JLabel
Pascal: begin ... end      Java: { ... }
```

16.5 Esercizi e compiti

1. **Hello World in Lazarus e in Java** (il testo del 28/10)
   1.1 Apri la lezione e segui le schede, una alla volta (la pagina cambia durante la lezione: ogni tanto premi F5): https://nicolaregge-pulse.github.io/corso-informatica/verifica/?b=3inf-hello-lazarus-java
   1.2 In Lazarus: progetto nuovo Documenti, Lazarus, Saluto; scrivi il nome, premi il bottone, compare il saluto.
   1.3 In Java, nel browser (JavaFiddle, senza installare niente): Hello World.
   1.4 Apri il Documento del compito (è solo tuo) e metti: codice Lazarus e screenshot, codice Java e screenshot, una frase con parole tue.
   1.5 Premi «Consegna». Bloccato? In fondo alla lezione c'è l'assistente: chiedi lì.
   1.6 Il 29/10 il testo del compito è stato aggiornato con il lavoro della lezione successiva (capitolo 17).
   1.7 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/MjU1MTAyOTQzNjBa/details
   1.8 Che cosa si consegna: il Documento con codici, screenshot e frasi.

17 Giovedì 29/10/2026 — Lazarus e Java a confronto

17.1 Quando

1. Giovedì 29/10/2026, 2 ore, Laboratorio.

17.2 Che cosa abbiamo fatto

1. Il percorso guidato «Lazarus e Java a confronto»: si finisce Hello World in Eclipse (percorso facilitato di 3 passi per chi deve ancora scompattare o aprire Eclipse) e si completa il Documento del compito con gli screenshot, la tabella del confronto e 3 frasi. Per chi non è riuscito a far girare Java, un Documento con le perplessità e il confronto.

17.3 La teoria

1. **Il Documento alla fine** ha 7 sezioni: 1 il mio codice Lazarus; 2 il mio programma Lazarus che funziona (screenshot); 3 il mio codice Java; 4 il mio programma Java che funziona (screenshot di Eclipse); 5 con parole mie; 6 Lazarus e Java a confronto (la tabella); 7 in 3 frasi, che cosa cambia e che cosa resta uguale.
2. **La tabella del confronto** (sezione 6), con le domande che aiutano:
   2.1 ambiente: Lazarus e Eclipse (riga di esempio, già scritta);
   2.2 linguaggio: come si chiama la lingua del codice in Lazarus e in Eclipse;
   2.3 dove si scrive il codice: in quale file e in quale parte della finestra;
   2.4 come si avvia: quale tasto in Lazarus (F9), quale bottone in Eclipse (Run);
   2.5 la finestra e il bottone: come si chiamano in Lazarus e nel codice Java (le righe con new);
   2.6 il punto e virgola e le virgolette: dove metti il `;` in tutti e due; ShowMessage usa gli apici singoli, Java le virgolette doppie;
   2.7 begin … end contro `{ }`: con che cosa apri e chiudi un blocco;
   2.8 i file del progetto: Lazarus .lpi .pas .lfm; Java .java e, dopo il Run, .class.
3. **Lo screenshot:** `Win + Shift + S`, trascina un rettangolo intorno alla finestra, poi `Ctrl + V` nel Documento. Se avevi già consegnato: «Annulla consegna», completa, poi di nuovo «Consegna».
4. **Come viene valutato:** si parte da 40; +10 lo screenshot di Hello World in Lazarus (sezione 2); +10 lo screenshot in Eclipse o Java (sezione 4); +10 la tabella del confronto completa, con parole tue (sezione 6); +10 le 3 frasi finali (sezione 7). Massimo 80. Righe vuote, copiate dall'esempio o senza senso non contano.

17.4 Da ricopiare sul quaderno

```
LAZARUS: Object Pascal, F9, begin ... end, apici singoli '...'
ECLIPSE: Java, Run, { ... }, virgolette doppie "..."
tutti e due: ; alla fine dell'istruzione
```

17.5 Esercizi e compiti

1. **Hello World in Lazarus e in Java** (il testo aggiornato il 29/10)
   1.1 Testo del compito: oggi 29/10, Lazarus e Java a confronto.
   1.2 Apri il percorso guidato e fai i passi uno alla volta; alla fine di ogni passo premi «Fatto»: https://nicolaregge-pulse.github.io/corso-informatica/3inf-confronto/ (con le righe in bangla: https://nicolaregge-pulse.github.io/corso-informatica/3inf-confronto/?pc=14)

1.3 Parte 1 (passi 1-3): finisci Hello World in Eclipse. Chi deve ancora scompattare Eclipse fa il percorso facilitato di 3 passi.
1.4 Parte 2 (passi 4-9): nel Documento del compito completa le sezioni 2 e 4 (gli screenshot) e aggiungi in fondo la sezione 6 (la tabella del confronto) e la sezione 7 (3 frasi).
1.5 Passo 10: premi «Consegna». Se hai già consegnato ieri: «Annulla consegna», completa, poi di nuovo «Consegna».
1.6 Questo Documento viene valutato: parte da 40; +10 lo screenshot di Hello World in Lazarus (sezione 2); +10 lo screenshot di Hello World in Eclipse o Java (sezione 4); +10 la tabella del confronto completa, scritta con parole tue (sezione 6); +10 le 3 frasi finali (sezione 7). Massimo 80. Righe vuote, copiate dall'esempio o senza senso non contano.
1.7 A che punto sei: https://nicolaregge-pulse.github.io/corso-informatica/3inf-lazarus/ ; la guida di Eclipse passo passo: https://nicolaregge-pulse.github.io/corso-informatica/3inf-eclipse-guida/
1.8 Bloccato? In fondo al percorso c'è «Chiedi aiuto all'assistente del Prof Regge»: scrivi lì la tua domanda.
1.9 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/MjU1MTAyOTQzNjBa/details
1.10 Che cosa si consegna: il Documento con le sezioni 1-7.

2. **NON SONO RIUSCITO AD ESEGUIRE HELLO WORLD JAVA**
   2.1 Compito aggiunto dal prof durante la lezione per chi non riusciva a far girare Java; il suo testo non è conservato nell'archivio del corso. Il lavoro: un Documento con le perplessità (che cosa hai provato, dove si è fermato, che errore compariva) e il confronto tra Lazarus e Java (la tabella della sezione 6).
   2.2 Link: https://classroom.google.com/c/MjUzMjQ1NzQ2NDBa/a/MjU1MTIzMDM4MTFa/details
   2.3 Che cosa si consegna: il Documento con le perplessità e il confronto.

18 Riepilogo di tutti i compiti

La tabella elenca i 18 compiti dati su Classroom alla 3INF fino al 30/10/2026, in ordine di data. Il capitolo dice dove trovare la teoria e le istruzioni.

N.	Data	Compito	Che cosa si consegna	Capitolo
1	25/09	GitHub	Documento «Git e Versioning» con le 3 parti	02
2	28/09	I tipi di file	Documento «I tipi di file» con le 5 parti	03
3	30/09	Esercitazione pagina HTML	Documento: screenshot, codice e riflessione	04
4	30/09	CSS in un file separato	il Documento aggiornato con HTML e stile.css	04
5	02/10	Pagina HTML	la pagina «Mi presento» con i due file	05
6	05/10	Pagina personale HTML	Documento: link, HTML, CSS, screenshot, spiegazione	06
7	07/10	Compito 3 — Il sito migliorato	Documento con le 5 parti e la riflessione	07
8	08/10	Compito 4 — Il treno blindato della 3INF: le mie Pull Request	Documento con PR-1, PR-2, PR-3, PR-4 e che cosa ho imparato	08
9	12/10	Versioning: il tuo Documento nell'albero delle versioni	il lavoro della pagina, fase 1 e fase 2	09
10	14/10	Il merge: le vostre richieste entrano nel gioco	almeno 3 richieste (PR1, PR2, PR3)	10
11	14/10	Ripasso e verifica: versioni, Pull Request, Agile	ripasso e verifica sulla pagina	10
12	14/10	VERIFICA: clicca qui	verifica su Moduli inviata	10
13	15/10	Sei il Product Manager: un team Agile al lavoro	v3.0, v3.1 e 3 risposte con «Consegna al prof»	11
14	19/10	Il mio sito diventa il mio portfolio	la pagina competenze.html online e la verifica	12
15	21/10	OGGI: il mio sito in ordine, più bello, e lavoro in coppia	sito in ordine, Pull Request in coppia, quiz su Moduli	13
16	22/10	OGGI: recupero — il mio sito personale funziona	il sito di 3 pagine e il quiz	14
17	28/10	Hello World in Lazarus e in Java	Documento: codici, screenshot, tabella del confronto, frasi	16
18	29/10	NON SONO RIUSCITO AD ESEGUIRE HELLO WORLD JAVA	Documento con le perplessità e il confronto	17

19 Glossario

Parola	Che cosa vuol dire
@keyframes	un'animazione CSS che parte da sola
Agile	modo di lavorare a piccoli passi, con versioni frequenti che funzionano
backlog	la lista ordinata delle richieste
branch (ramo)	una strada di lavoro parallela
bytecode	il codice Java tradotto per la Java Virtual Machine
changelog	il registro dei cambiamenti, versione per versione
classe (CSS)	un nome dato a un pezzo di pagina; nel CSS si scrive con il punto
clone	scaricare la prima volta un progetto sul proprio computer
collaboratore	chi ha il permesso di modificare il repository di un altro
commit	un salvataggio nella storia, con un messaggio
compilato	tradotto una volta in un programma pronto (.exe)
conflitto	due rami che cambiano la stessa riga: si sceglie quale tenere
CSS	il linguaggio dello stile: colori, spazi, animazioni
Eclipse	l'ambiente di sviluppo (IDE) per Java
fork	la copia di un progetto intero che cresce per conto suo
fps	fotogrammi al secondo di un video
Freemium	base gratis, funzioni in più a pagamento
Git	lo strumento che tiene la storia di un progetto
GitHub	il sito che tiene i repository online
GitHub Pages	il servizio gratis che trasforma un repository in un sito
History	la lista dei commit di un file su GitHub
href	l'indirizzo dove porta un link
HTML	il linguaggio a tag delle pagine web: il contenuto
IDE	ambiente di sviluppo: editor, compilatore ed esecuzione insieme
Java Virtual Machine	il programma che esegue il bytecode Java
major / minor	primo numero della versione (cambio grosso) e secondo (piccola aggiunta)
merge	unire un ramo o una Pull Request dentro il progetto
new	in Java crea un oggetto (una finestra, un array)
Object Pascal	il linguaggio di Lazarus
PaaS	Platform as a Service: una piattaforma online già pronta
portfolio	la pagina che dice che cosa sai fare, per lo stage
priorità	valore diviso (peso più rischio): dice che cosa fare prima
Product Manager	chi decide l'ordine del backlog e che cosa entra nello sprint
Pull Request	proposta di modifica: titolo, cosa cambia, dove, perché
push / pull	mandare / prendere le versioni da e verso GitHub
refresh rate	quante volte al secondo si aggiorna lo schermo (Hz)

Parola	Che cosa vuol dire
release	una versione finita e pubblicata, con un numero
repository	la cartella del progetto con tutta la sua storia
retrospettiva	l'incontro in cui la squadra decide come lavorare meglio
selettore	nel CSS dice a che cosa si applica la regola
sprint	giro di lavoro breve con una capacità fissa e una versione alla fine
Swing	la parte di Java che fa le finestre
tag (HTML)	un'etichetta tra parentesi angolari: apre e chiude un pezzo
tag (Git)	la bandierina con il numero di versione su un commit
transition	un cambiamento morbido nel CSS
upload	caricare i file dal PC a GitHub
workspace	la cartella in cui Eclipse salva i progetti

20 Changelog

1. **v1.0 (30/10/2026)**: prima versione. Le lezioni della 3INF dal 23/09 al 29/10/2026 (17 capitoli), con la teoria in breve, le foto della lavagna, le righe del quaderno, il testo completo dei compiti dati su Classroom e la tabella di riepilogo; glossario. Solo italiano: le righe in bangla restano nelle pagine delle lezioni.